ksat-ports

Shadowing Methodology — how we port SAT solvers to Kotlin, bulletproof

Living document · grows with each solver

Contents

Purpose 3
The shadowing procedure (per solver) 3
Why step-by-step (not just verdict) matters 4
Comparison levels (pick the strongest that holds) 4
Test-CNF strategy 4
Lessons learned (append-only; re-check earlier ports on each new entry) 5
Per-solver status checklist 10

Purpose

We port several established C/C++ SAT solvers to Kotlin Multiplatform, in order: **MicroSAT** → **MiniSat** → **CaDiCaL (core)** → **kissat** → **CaDiCaL (full)**. Each port must be proven to behave **identically** to its original — ideally **step by step**, not just in the final SAT/UNSAT verdict. This document is the reusable procedure and the shared **lessons learned**, so that what we learn on one solver is applied to the next, and — the bulletproof part — **earlier ports are re-checked against later lessons**.

> **The bulletproof loop**
> Forward: apply every lesson from solver N to solver N+1 before writing it. Backward: whenever a new lesson emerges (on any solver), **revisit all earlier ports** and confirm they already satisfy it, or fix them. A lesson is not "done" until every port in the repo honours it. The checklist in the last section tracks this per solver.

> **CaDiCaL core port removed (2026-10-22)**
> The CaDiCaL core port (class `CaDiCaL` with all inprocessing off) and its hand-transcribed reference `shadow/cadical-c/cadical_trace.cc` (with `regen_golden_cadical*.sh`, `validate_upstream_cadical.sh` and the `golden-cadical*` tests) were deleted on the owner's decision. The full 1:1 port of CaDiCaL 2.1.3, checked against the instrumented real upstream in `shadow/cadical-full/`, replaced it and took over the name `CaDiCaL` (see `cadical-full-port.md`). The "CaDiCaL core" lessons below are kept as history; they still hold for the full port.

The shadowing procedure (per solver)

1. **Copy, never edit upstream.** The originals live read-only in `../other_repos/`. Copy the source into `shadow/<solver>-c/` as `<solver>_orig.c` (verbatim) and `<solver>_trace.c` (instrumented). We never modify `other_repos/`.
2. **Instrument the C reference.** Add trace emission at the decision-relevant events only, gated by an env var (`LSTRACE`) so normal runs stay clean. No algorithm change — only `printf`s. Shared trace vocabulary: `DECIDE`, `ASSIGN … reason=…`, `CONFLICT`, `RESTART`, `REDUCE`, `RESULT SAT|UNSAT`. Extend the vocabulary only when a solver has a genuinely new event; keep older events stable.
3. **Port to Kotlin faithfully.** Translate 1:1, favouring correspondence over idiom. Flat C arrays with pointer arithmetic (`int* DB`) become `IntArray` with integer indices; arrays indexed by signed literals become size-`2n+1` arrays with a `+n` offset. Emit the **same** trace from the same points.
4. **Diff traces byte-for-byte.** The shadow test runs both on the same CNFs and asserts the Kotlin trace equals the C trace line-for-line, the verdict matches, and (on SAT) the returned model satisfies the CNF. Any divergence is a port bug.
5. **Cover every code path.** Small CNFs don't trigger `RESTART`/`REDUCE`. Prefer a **test-only threshold knob** (mirrored in C and Kotlin, defaulting to the upstream value) that lowers the trigger so a **small** instance's trace contains those lines — cheaper and faster than hunting for a big/hard instance. Either way the hand-translated paths must be trace-covered; they are the likeliest to hide bugs.
6. **Keep an independent upstream oracle.** The instrumented reference is hand-edited, so also check it against genuine upstream (verbatim recompile for MicroSAT; a real `minisat` binary when present for MiniSat) on the verdict + conflict count, to catch a shared transcription bug that trace equality alone would miss.
7. **Benchmark runtime.** Confirm the port is not just correct but comparably fast (same order of magnitude as C; a small constant factor slower is expected on the JVM). Never claim performance without measuring.

Why step-by-step (not just verdict) matters

Matching only SAT/UNSAT hides bugs: a port can reach the right answer by a different search and still be subtly wrong (and diverge on the next instance). Trace equality pins down the **same decisions, propagations, conflicts, restarts in the same order**. It is a far stronger contract — and it is **achievable** when the solver's decision heuristic is deterministic and integer-based.

> **Caution**
> Step-by-step equality is only realistic when the heuristic has no language-dependent nondeterminism. **MicroSAT** uses integer VMTF and integer moving-average restarts → fully reproducible, byte-for-byte. **MiniSat and up** use `double` VSIDS activities; floating-point rounding may differ across C, JVM, and Kotlin/Native, so a single differently-rounded activity flips a decision and the whole trace diverges from there. For those solvers we degrade gracefully (see next section) rather than pretend trace equality holds.

Comparison levels (pick the strongest that holds)

L1 — full decision trace every event identical, line-for-line. The gold standard. Target for integer-heuristic solvers (MicroSAT).

L2 — structural trace with tolerant fields compare the event **sequence** but allow controlled differences (e.g. treat VSIDS tie-breaks via a fixed rule, or compare decision **variables** while ignoring exact activity values). Use when L1 breaks only on float ties.

L3 — verdict + DRAT proof + model same SAT/UNSAT; on SAT the model satisfies the CNF; on UNSAT the emitted DRAT proof checks with `drat-trim`. Decouples correctness from decision order. The robust permanent gate for float-heuristic solvers, and a good secondary gate everywhere. DRAT + `drat-trim` is the **intended permanent L3 gate for UNSAT** (MicroSAT upstream has a DRAT-emitting variant); it is not yet wired here because `drat-trim` isn't installed on the build machine — this is the next hardening step. Until then the independent upstream **verdict** oracle (verbatim recompile / real `minisat`) is the UNSAT cross-check.

Always run the **strongest level that holds** for a solver, plus L3 as a safety net. Document per solver which level was achieved.

Test-CNF strategy

- **Tiny hand-built:** pinpoint specific mechanics (a forced unit, a single conflict, a parse-time UNSAT). Easy to reason about; first line of defence.
- **Pigeonhole PHP(n+1, n):** small, guaranteed UNSAT, forces conflict analysis — great for exercising the learning path deterministically.
- **Random 3-SAT near the phase transition** (clause/var ratio ≈ 4.26): a spread of SAT and UNSAT of tunable hardness; scale up until `RESTART`/`REDUCE` fire.
- **Game-derived CNFs:** the real workload — per-tile cardinality instances from lucky sweeper boards. Add these once the encoding exists, so the solver is tested on what it will actually see.

Keep every generator and every generated `.cnf` in the repo (never `/tmp`, never delete): they are the regression corpus.

Lessons learned (append-only; re-check earlier ports on each new entry)

Lesson
(MicroSAT) Kotlin forbids reusing one `var` name across two loops in a function where C reuses a single `for` index. This surfaced in `reduceDB`. Fix: distinct index names. **Re-check rule:** scan every port for C loops that reuse an index variable across sequential loops.

Lesson
(MicroSAT) C's `int* watch = &first[lit]` that later becomes `watch = DB + *watch` is a pointer that points **either** into the offset array **or** into `DB`. In Kotlin model it as an explicit indirection (a boolean "is it the first-array slot?" plus an index). This is the single most error-prone translation pattern in a watched-literal solver. **Re-check rule:** every watched-literal port must handle this dual-location pointer explicitly and have it trace-covered.

Lesson
(process) Small CNFs pass while leaving `RESTART`/`REDUCE` untested. Do not declare a port shadow-verified until a CNF whose C trace **contains** those events passes too. **Re-check rule:** every port's shadow suite must include a RESTART- and a REDUCE-triggering instance.

Lesson
(MiniSat, float VSIDS) `double` activities are IEEE-754 identically on C and the JVM, and if every activity update is transcribed in the **same evaluation order** (same `+=`, same rescale thresholds `1e100`/`1e-100`/`1e20`/`1e-20`, same `var_inc *= 1/var_decay`), L1 full-trace equality is **achievable**, not just L2 – the float worry only bites when two variables reach **bit-identical** activity and the heap tie-break then depends on insertion history. Keep the decision `Heap`'s `VarOrderLt` as strict `>` (not `>=`) and reproduce `percolateUp/Down` exactly so ties resolve the same way. The shadow test still degrades gracefully to L2 per instance and always enforces L3. **Re-check rule:** any float-heuristic port must transcribe activity arithmetic operation-for-operation and use the identical heap comparator + sift order.

Lesson
(MiniSat, arena translation) MiniSat's `RegionAllocator<uint32>` + `CRef` + packed `Clause` header become, cleanly, a set of parallel `IntArray`/`DoubleArray` columns indexed by an `Int` handle (`caLits[cr]`, `caSize[cr]`, `caAct[cr]`, `caLearnt[cr]`, `caMark[cr]`) – no byte-level header packing needed, because trace equality depends on the clause **values**, not their memory layout. Watched literals are two parallel lists per literal (`watchCref[lit]`, `watchBlk[lit]`) indexed by `Lit.x` (size `2n`). **Re-check rule:** arena ports may drop byte-faithful headers but must keep value-faithful clause contents and the exact watcher order.

Lesson
(MiniSat, model capture ordering) MiniSat copies the model **before** `cancelUntil(0)` runs at the end of `solve_`. Capturing after backtrack yields a mostly-`Undef` model and a spurious "model does not satisfy CNF". Capture the model at the `l_True` point, then backtrack. **Re-check rule:** every port must snapshot the satisfying assignment before any top-level cancel.

Lesson

(MiniSat, reduceDB sort) MiniSat's `reduceDB_lt` is a **non-total** order (binaries and activity mixed), and it is fed to a sort. If the C uses `std::stable_sort` and the Kotlin uses `sortedWith`/Timsort, incomparable clauses can land in different orders on large instances -> a genuine L1 divergence **inside** reduceDB. Fix: use the **same** stable sort with the **same one-directional predicate** on both sides (here a plain stable insertion sort with `reduceLt(x,y)`), so "equal" clauses keep input order identically. **Re-check rule:** any port that sorts with a non-strict-weak comparator must match the C sort algorithm, not just the comparator.

Lesson

(MiniSat, clause-activity float width) Upstream stores **clause** activity in a 32-bit `float` (`Clause::act`), not a `double` — and that 32-bit rounding is observable: `reduceDB` prunes learnt clauses by activity, so a differently-rounded value can keep a different clause and diverge the trace. A Kotlin `DoubleArray` silently over-precises it. Fix: make activity precision **configurable** (`ActivityPrecision.FLOAT32` = round every activity write via `toFloat().toDouble()` to match C bit-for-bit for shadowing; `FLOAT64` = full precision for standalone use). Default to FLOAT32 for the shadow build. Verified: with FLOAT32 the port stays L1 on all instances incl. the reduceDB-heavy php_6_5/7_6/8_7. **Re-check rule:** for every float-heuristic port, match the C's **storage width** per field (float vs double), not just the arithmetic order — and expose the width as a knob.

Lesson

(process, don't silently downgrade L1) A shadow test that accepts either L1 or L2 will report a real byte-for-byte regression as a passing "L2" and nobody notices. Once a port reaches L1 on an instance, **gate on it**: fail the test if that instance later drops to L2. The "byte-for-byte" claim is only real if it is asserted at that granularity, not merely printed. **Re-check rule:** every port with L1 instances must have a hard L1 regression gate for them.

Lesson

(process, cover rare paths with a TEST-ONLY threshold knob) When a code path only fires past a conservative internal threshold (MicroSAT's `REDUCE`/`reduceDB` needs `nLemmas > maxLemmas` with the upstream default `maxLemmas = 2000`), do **not** reach for a giant slow CNF to trip it — that bloats the fast suite. Instead expose the threshold as an optional, default-upstream knob on **both** sides and set it low in one dedicated test: Kotlin `MicroSat(numVars, maxLemmasInit = 3)`, C `LSMAXLEMMAS=3` read in `initCDCL`. With the same low value the two stay byte-for-byte identical, and a tiny existing UNSAT instance (php_5_4/php_6_5) now trace-covers the dual-location watch-pointer compaction in `reduceDB` — the exact path a static-only review can't vouch for. The default equals upstream, so production behaviour is unchanged. **Re-check rule:** for any port whose `RESTART`/`REDUCE`/GC path hides behind a conservative threshold, add a mirrored test-only knob (Kotlin ctor param + C env var) rather than an expensive instance, and assert the rare event actually appears in the golden.

Lesson

(process, verbatim-upstream verdict oracle catches transcription bugs) The instrumented `*_trace.{c,cc}` references are hand-edited copies; if the edit (or the copy) silently changed the algorithm, the port can match the trace and still be wrong — `Kotlin ≡ trace ≡ wrong`, with nothing to notice. Add an **independent** oracle that never touches `other_repos/`: compile the **verbatim**

upstream that's already copied into the repo (`microsat_orig.c`) and diff its `s SATISFIABLE`/`s UNSATISFIABLE` verdict **and** its `c statistics … conflicts: N` count against the instrumented reference on every CNF (`shadow/tools/validate_upstream_microsat.sh`). Any divergence means the reference drifted from real upstream. When upstream is too heavy to build in-repo (MiniSat needs `mtl`/`zlib`/its build system), degrade to an **optional** verdict oracle against a real binary if present on `PATH` (`validate_upstream_minisat.sh`: run `minisat` when available, skip cleanly otherwise), and document the manual install. **Re-check rule:** every solver keeps a verbatim/real-upstream verdict oracle so a transcription bug in the instrumented reference can't hide behind trace equality. The permanent L3 gate for UNSAT is DRAT + `drat-trim`; wire it once `drat-trim` is available (next hardening step).

Lesson

(CaDiCaL core, score width — the FLOAT32 knob points the OTHER way) Where MiniSat needs `ActivityPrecision.FLOAT32` to match C (its `Clause::act` is a 32-bit `float`), CaDiCaL's core stores EVSIDS scores (`stab`) **and** the glue EMAs (`glue_fast`/ `glue_slow`) as `double`. So here the byte-for-byte configuration is `ActivityPrecision.FLOAT64` (the default), not FLOAT32. Same re-check rule, opposite answer: match the C's **storage width per field**, don't assume the previous solver's width carries over. The knob still exists (FLOAT32 rounds every score write) for a hypothetical float build, but the shadow build uses FLOAT64 and the hard L1 gate is set on that.

Lesson

(CaDiCaL core, EMA / EVSIDS evaluation order is load-bearing at L1) Byte-for-byte equality on a `double`-heuristic solver needs more than the same storage width — the **evaluation order** of every floating-point update must be transcribed verbatim. The ADAM-style bias-corrected EMA (`old_biased`/`delta`/`scaled_delta`/`new_biased`, then the `1 - new_exp` divisor) and the EVSIDS `score + score_inc` / `score_inc *= 1e3/factor` updates are each ported statement-for-statement in the same order as `ema.cpp`/ `score.cpp`, because reassociating them would round differently and perturb which variable wins a heap tie — silently downgrading L1 to L2. **Re-check rule:** for any float-heuristic port, mirror the arithmetic **expression tree and order**, not just the final formula; the hard L1 gate is what catches a reordering regression.

Lesson

(CaDiCaL core, non-total-order stable sorts — same lesson, three more sites) CaDiCaL has several `std::sort`/`std::stable_sort` calls whose comparators are not strict total orders over the **values** being compared: reduce candidates by `(glue, size)` (ties keep input order — `std::stable_sort`), and the learned clause by trail order and by `(level, trail)` for the driving-clause watches. The `(level, trail)` and trail keys **are** strict total orders over distinct variables, so any correct sort reproduces C's result; but the reduce `(glue, size)` predicate has real ties, so it must be a **stable** sort to match `std::stable_sort` byte-for-byte. Ported with stable insertion sorts throughout (same fix as MiniSat's `reduceDB`). **Re-check rule:** audit every comparator in a port for ties; use a stable sort wherever the C used `stable_sort` or the key is not a strict total order over the compared values.

Lesson

(CaDiCaL core, cover RESTART/REDUCE without a knob when the defaults are aggressive) The rare-path coverage lesson (test-only threshold knob) still applies, but CaDiCaL's **upstream**

defaults are already aggressive: `restartint = 2`, `reduceint = 25`, `stabilizeinit = 1000`. On the existing php_6_5/php_7_6/php_8_7 instances these fire RESTART and REDUCE at the stock values, so the golden traces cover both paths with NO knob and while staying byte-for-byte (the C reference has these as fixed `const`s, not env-readable, so lowering them would need patching C too and would drift from "upstream defaults"). The Kotlin ctor still **exposes** `reduceIntInit`/ `restartIntInit`/ `stabilizeInit` for standalone experiments, defaulting to the upstream values. **Re-check rule:** prefer the existing suite when a solver's stock thresholds already trip the rare path; only reach for a mirrored knob (and a matching C env read) when they don't.

Lesson

(kissat core, score width — the double answer again, but a DIFFERENT heap tie-break) Like CaDiCaL, kissat stores its decision scores and the score increment as `double` (`heap.score` is `double *`, `solver->scinc` is `double`, `bump.c`), and the glue EMAs as `double` (`smooth.c`). So the byte-for-byte configuration is again `ActivityPrecision.FLOAT64` (the default), **not** FLOAT32. The knob still exists for a hypothetical float build. BUT the heap itself differs from CaDiCaL in a way that matters at L1: kissat's max-heap (`inlineheap.h` `bubble_up`/`bubble_down`) is keyed **purely by the double score** and breaks ties by POSITION (insertion order in the heap array) — there is **no** index tie-break (CaDiCaL's `heap_less` adds `a < b`). `bubble_up` stops on `score[parent] >= idx_score`; `bubble_down` prefers the left child on an equal-score sibling (`sibling_score > child_score`) and stops on `child_score <= idx_score`. The port reproduces those exact comparisons; using CaDiCaL's index tie-break here would silently pick a different variable on a score tie and diverge the trace. **Re-check rule:** match not only the score **width** per field but the heap comparator's **tie-break rule** — it is solver-specific (index tie-break in CaDiCaL, positional tie-break in kissat).

Lesson

(kissat core, unsigned literals + XOR watch trick + unsigned right shift) kissat is unsigned-heavy: internal literals are `lit = 2*idx + sign`, `NOT(lit) = lit ^ 1`, `IDX(lit) = lit >> 1` (a **logical** right shift on `unsigned`). `values[]` is indexed by literal with `values[lit] = -values[NOT lit]`. The watched-literal "other" is recovered with `other = lits[0] ^ lits[1] ^ not_lit`. In Kotlin, translate `IDX` as `lit ushr 1` (NOT `shr`), keep literals as non-negative `Int`, and mask the reduce rank's `~size` / `~glue` to 32 bits (`x.toLong().inv() and 0xffffffffL`) before packing `~size | (~glue << 32)` so the sign extension of `Int.inv()` doesn't corrupt the key. Special reason values `DECISION_REASON = UINT_MAX`, `UNIT_REASON = UINT_MAX-1`, `INVALID_REF/LIT/LEVEL = UINT_MAX` become sentinel `Int` constants (`-1`, `-2`); the heap "discontained" position `UINT_MAX` likewise. **Re-check rule:** every unsigned port uses `ushr` for index extraction, keeps literals non-negative, and masks before any left-shift pack so `Int.inv()` sign bits can't leak into a 64-bit key.

Lesson

(kissat core, watches are a flat variable-stride word list — 1 word binary, 2 words large) Unlike CaDiCaL's parallel `(cref, blit, bin)` watcher columns, kissat packs a watch list as a flat vector of 32-bit words: a **binary** watch is ONE word (blocking lit + a binary tag bit), a **large** watch is TWO words (a blocking-literal word then a raw clause reference word). Iteration must read the head word, test the binary tag, and only then consume the second (reference) word for large watches. The port keeps this exact encoding in a single `IntArrayList` per literal (`(lit<<1)|1` binary, `(lit<<1)` blocking + raw ref large) so propagation's in-place compaction (`q -= 2` to drop a large watch, delayed re-watching of the replacement) advances by the same strides as the C. Binary clauses are **watch-only** — they never get an arena clause. **Re-check rule:** a variable-

stride packed watch list must be iterated by reading the tag first and advancing 1-or-2 words to match; don't "normalise" it to fixed-width watchers or the compaction indices diverge.

Lesson

(kissat core, analyze is a two-stage beast: conflict-level reuse THEN frame-based 1-UIP) kissat's `analyze` first runs `one_literal_on_conflict_level`: it finds the conflict level and, when exactly one literal sits on it, **reuses the conflict clause itself as the driving clause** (after moving the two highest-level literals to the front and re-watching), with no new clause learned — this is not chronological backtracking, it's a legitimate CDCL shortcut and must be ported faithfully. Only otherwise does it run the first-UIP deduction (`deduce.c`), which counts unresolved current-level literals via per-decision-level `frame.used` counters (not CaDiCaL's single `open` counter), pulls each newly-seen lower level into a `levels` list, then `sort_deduced_clause` rebuilds the learned clause **highest-decision-level first** using those per-level `used` offsets, then recursive `minimize`. A `conflict_level == 1` special case (`analyze_failed_literal`) learns a batch of units. All of this is integer/pointer-deterministic and was transcribed statement-for- statement. **Re-check rule:** when a solver's analyze has a conflict-reuse fast path and a level-frame 1-UIP, port BOTH paths and the level-sorted rebuild exactly — the fast path fires constantly and skipping it diverges immediately.

Lesson

(kissat core, what to disable to keep the core deterministic) kissat's **upstream defaults** enable several things that are neither inprocessing nor part of the plain CDCL core but do perturb the search path: chronological backtracking (`chrono=1`, `chronolevels=100`), on-the-fly self-subsumption/strengthening (`otfs=1`), extra clause shrinking beyond recursive minimize (`shrink=3`), reason-side literal bumping (`bumpreasons=1`), eager subsumption of the last few learned clauses (`eagersubsume=4`), random decisions (`randec=1`), and jump-reasons. The core milestone turns all of these OFF (documented in the C header and the Kotlin KDoc), exactly as the (removed) `cadical_trace.cc` disabled CaDiCaL's `chrono`/`otfs`/`shrink`. With `chrono=0`, `kissat_determine_new_level` degenerates to "always return the jump level", which is what makes backtracking reproducible. **Re-check rule:** for each new Biere solver, audit the option defaults for search-path-perturbing (but non-inprocessing) features and disable them for the core shadow, listing each one.

Lesson

(kissat core, stabilize/mode schedule is TICK-based upstream — a known approximation risk) kissat toggles focused/stable mode on a schedule driven by **ticks** (a cache-line-estimate counter incremented inside propagation, `mode.c`/`kimits.c`), not on a clean conflict count. Reproducing the tick counter byte-for-byte would require modelling kissat's cache-line accounting in `proplit.h` (`kissat_cache_lines`), which is an implementation detail unrelated to the CDCL logic. The reference and port therefore approximate the mode switch with a **conflict-based** quadratic schedule (`stableinit * count²`), matching CaDiCaL's stabilize shape, and expose a `stableInit` knob (mirrored by the C `LSSTABLEINIT`). This is the single most likely source of a late-instance L1/L2 divergence and is flagged for the orchestrator: if a hard instance drops to L2 around a mode switch, the fix is to either (a) transcribe the tick counter too, or (b) accept L2 across the switch and keep the hard L1 gate on the pre-switch (focused) prefix. **Re-check rule:** when a solver's phase schedule keys on a hardware-ish counter (ticks/cache-lines), decide up front whether to model it exactly or approximate, and record which — don't silently assume the approximation stays L1.

Per-solver status checklist

Solver	Ported	Level	RESTART/ REDUCE covered	Benchmarked
MicroSAT	yes	L1 (full trace), verified 15 CNFs + verbatim-upstream verdict oracle	**yes** — REDUCE trace-covered via `maxLemmasInit`/ `LSMAXLEMMAS` knob on php_5_4/ php_6_5	yes (0.8–6 ms/inst)
MiniSat	yes	**L1 verified + hard-gated** (FLOAT32 activity)	yes (php_6_5/7_6/8_7)	yes
CaDiCaL core	removed 2026-10-22	was L1 vs `cadical_trace.cc`; replaced by CaDiCaL full	—	—
kissat core	yes	**L1 verified + hard-gated** (FLOAT64, positional heap tie-break)	yes (php_7_6/8_7; php_8_7 = 146k lines)	yes; + real upstream `kissat` oracle agrees on all 15 CNFs
CaDiCaL full	yes	**L1** vs instrumented real upstream 2.1.3 (`cnf/`, `cnf-full/`, `cnf-assume/`; plain + default; proofs byte-identical)	yes (upstream defaults)	not yet (`benchmark_cadical.sh` retargeted 2026-10-22)

Update this table as each port lands. When a new lesson is added above, walk the “yes” rows and confirm each still complies.

> **Note**
> **(Historical: the core port and these scripts except `benchmark_cadical.sh` were removed 2026-10-22.) (CaDiCaL core — verification pending, blocked on disk space, 2026-09-01)** The CaDiCaL core port, its Gradle module, the shadow test (with the hard L1 gate and the RESTART/REDUCE coverage test), and all three tool scripts (`regen_golden_cadical.sh`, `validate_upstream_cadical.sh`, `benchmark_cadical.sh`) are in place and mirror the MiniSat harness 1:1. The verify+harden run (build C golden traces → `:cadical:compileKotlinJvm` → `:cadical:jvmTest` → benchmark) could **not** be executed: the machine’s root disk was at 100 % (≈49 MiB free), so every compiler / JVM / Gradle / trace-binary invocation failed with `ENOSPC`. Static spot-check of the port vs `cadical_trace.cc` (EMA update order, EVSIDS bump/rescale, score-write rounding) matched. The remaining L-level, golden traces, RESTART/REDUCE coverage and benchmark numbers must be produced by re-running the pipeline once disk space is freed.

> **kissat core — VERIFIED L1**
> The kissat core port is **verified L1 (byte-for-byte) on all 15 CNFs**, including php_8_7 (146k trace

lines) with RESTART+REDUCE. Field width: **FLOAT64** (double scores/EMAs, like CaDiCaL). The two flagged risks did **not** bite on the corpus: the **tick-based mode schedule** (approximated with a conflict schedule) stays byte-identical here, and the **reduce rank-tie stable sort** matches. A real upstream `kissat` binary agrees on all 15 verdicts. Hardest port so far (39k lines of unsigned/bit-packed C, kitten + all inprocessing excluded); it transcribed faithfully and compiled + shadowed on the first run. If a future harder instance drops to L2 near a mode switch, transcribe the tick counter (documented in the lessons).